iBPM
Campus model & Solana integration

The committee runs the platform. Makers are standing appointments. There is no rotation or cycle settlement.

1. Who can do what

Role	Capabilities
Admin committee	Markets, resolution, membership, maker appointments, funding, rewards, pause and upgrades. Sensitive actions need multiple approvals.
Market maker	All trader actions, plus add/remove liquidity using persistent maker capital.
Trader	Buy, sell, redeem winnings, submit market ideas and view. Must be enrolled.
Observer	Read-only. No enrollment needed to read chain state. No starting balance or trading.

We follow the permission matrix: only appointed makers provide liquidity. Committee membership does not automatically grant a personal trading account. There is no separate teacher or resolver tier.

2. Keep the balances separate

Personal GIES: the participant trades from their wallet token account. The configured starter is granted once; the default is 5,000 GIES.

Maker capital: a separate program-controlled desk funds liquidity. The committee appoints a maker, then funds the desk. The default policy size is 15,000 GIES. Top-ups and clawbacks are explicit; the balance is never reset by a cycle.

Market vault: holds collateral backing YES/NO claims. Winning claims redeem for 1 GIES each. Outcome claims are stored in position accounts, not separate SPL token mints.

Trading fees benefit LP reserves. Fee-matched rewards are separate payouts approved by the committee, with a remaining emissions budget and a one-time claim ID. Rewards start disabled until configured.

What is ready, and what comes next

Campus permissions, the Solana program, the shared client and the generated IDL are implemented. The current web app still defaults to PostgreSQL. The frontend owner needs to connect wallet screens, chain reads, quotes and the indexer before switching to Solana.

Connect the frontend

3. Start with the shared client

Import **@workspace/solana-client**. The frontend already has the workspace dependency. Use the checked-in IDL at **solana/idl/ibpm.json**; it matches this source version. Keep Anchor 0.32.1 and web3.js 1.98.4. Do not mix in a web3 v2 client.

The current program ID is `EiMn8BEsdfuxEedNGi2vr12cjbKhGpq5Bq5N2ecCdnnM`. Use the same cluster, program address, binary and IDL. This address alone does not prove that this revision is deployed.

4. Link, enroll, fund

1. Keep campus login for the roster and off-chain account. Connect a wallet, then call **linkCampusWallet(token, publicKey, signMessage)**. The API verifies a short-lived message signature. A link proves ownership; it does not grant on-chain membership or money.

2. The committee enrolls the verified wallet and calls **grantStarter** once. For a maker, appoint the maker and separately call **allocateMakerCapital**.

3. Read the chain before enabling actions: no enrollment account = observer; **kind 0 = trader**, **kind 1 = maker**. Read committee membership from config. An inactive participant can still exit existing positions.

5. Build and confirm a trade

Create the Anchor provider from the connected wallet, then use these helpers. The complete setup is in **docs/integration.md**.

```
const client = createClient(provider);
const ix = await client.buy(
  wallet.publicKey, 0n, YES,
  gies("25"), gies(minTokensFromQuote)
);
const prepared = await prepareTransaction(
  connection, wallet.publicKey, [ix]
);
const signed = await wallet.signTransaction(
  prepared.transaction
);
const signature = await submitSigned(
  connection, signed, prepared, savePendingSignature
);
// Refetch market, position and wallet balances.
```

Use a fresh chain quote and an explicit minimum output. Buys and sells both enforce that signed floor. Keep all amounts in BN/bigint with six decimals; use gies("12.345678"), not floating-point multiplication. The chain market ID is not the PostgreSQL row ID.

Store the signature as soon as it is submitted. A confirmation timeout is an unknown result. Check that signature before offering a second submission. Show success only after confirmation and account readback.

Committee flow & cutover

6. Multiple people sign one transaction

The program supports 2-7 committee members. Routine actions require at least two signatures. Resolution, committee changes, capital clawbacks, rewards and upgrades use the sensitive threshold. Any one member can pause new risk; unpausing needs routine approval.

Build with **client.program.methods** and **.remainingAccounts(committeeSigners(payer, members))**. Members sign the **same transaction message**; public keys alone do not count. An expired blockhash requires fresh signatures. Private keys stay in member wallets.

Database-mode committee proposals are a separate approval mechanism. They do not count as signatures on a Solana transaction. Resetting a committee account password also needs committee approval.

7. Index the chain; keep app metadata off-chain

Use the chain for balances, collateral, trades and resolution. Keep titles, source links, roster/cohorts and presentation data off-chain. Map market metadata by **cluster + program ID + market public key**.

The **TradeExecuted** event supplies executed amounts, fees and reserves. Index by transaction signature and instruction/event location, backfill missed transactions, and reconcile to finalized state. Do not treat optimistic UI balances as settled balances.

For rewards, the committee reviews the fee report and calls **payReward** with its evidence hash. The program checks the match and budget. Reject overlapping reports even if claim IDs differ.

8. Before enabling Solana mode

Owner	Remaining step
Frontend	Wallet connection/linking; chain membership and balances; quote/slippage UI; trade, LP and redemption flows; pending/error states.
Integration	Market metadata mapping, event history/indexer, backfill and finalized-state reconciliation.
Committee / deployer	Deploy this build, initialize members/thresholds, transfer upgrade authority to config PDA, and verify readback. Configure economics and enroll/fund participants.

The deployer must transfer program upgrade authority to the config PDA. Commands are in the full handoff; the local test exercises the transfer and committee-approved upgrade.

Only after the wallet integration is ready, set **MARKET_BACKEND=solana**. Legacy financial endpoints then return 503, which prevents the app from using a second database ledger. Use a fresh campus database; beta balances and old on-chain accounts need a reviewed migration.

Validation: web build/type checks, isolated API tests, Rust/client tests and 13 local-validator scenarios pass. No public-cluster deployment, production migration or live authority transfer was performed. Browser automation was unavailable for rendered UI review.

Full account lists, code examples and operator steps: docs/integration.md. Working transaction examples: **solana/tests/ibpm.ts**.